



Welcome to this exciting three-part series where you will learn to create a complete monster collector-style game using the Godot engine. Throughout this course, you will build a game featuring an explorable overworld, battles with trainers and wild monsters, and a quest system. This project will combine many advanced Godot concepts, such as state management and how to work with data in a more complex game. Your instructor for this course will be Christian.

What We Will Build in Part One

In the first part of this series, we will focus on creating the game's overworld. This will involve building multiple levels and populating them with trainers. We will implement dialogue options for each trainer and, most importantly, establish a system for storing data globally. This ensures that the state of each trainer, such as whether they have been defeated, remains consistent even when you move between different scenes or reload a level.

Course Learning Goals

This course is designed to help you explore more advanced functionalities within Godot. The key learning objectives include:

- **State Management:** Learn how to create levels where enemies and other elements remember their state. This is a crucial concept for almost any complex game, ensuring that if a trainer is defeated, they remain defeated permanently.
- **Data Organization:** Practice working with large amounts of data. We will manage information for numerous levels, extensive trainer dialogue, and, in later parts, detailed monster data, all while keeping it organized and manageable.
- **TileMap Mastery:** Gain in-depth experience with Godot's TileMap layers, including implementing animations, handling collisions, and building detailed environments.

Prerequisites for This Course

It is important to note that this is an advanced course. To succeed, you should have a solid understanding of Godot and GDScript. This course will move quickly, and a foundational knowledge is essential. Ideally, you should have:

- Experience creating a few smaller games in Godot.
- Familiarity with programming concepts such as inheritance, enums, tweens, and classes.

If you meet these requirements, this course will be an excellent opportunity to practice and solidify your advanced skills. At Zenva, we offer a wide range of courses to help you learn, whether you are just starting out or looking to expand your knowledge. Our courses are versatile, allowing you to learn through video tutorials, lesson summaries, and the included project files. We look forward to seeing you in the course!

Which version of Godot should you use for this course?

Technology changes rapidly, so to make sure you have an optimal learning experience, we recommend using **Godot 4.5** for this course – the latest stable release.

Please make sure NOT to use newer versions of the software than what we recommend, as the course material provided might not work as expected.

How to Install Version 4.5

You can download the most recent version of Godot by heading to the Godot website (<https://godotengine.org/>) and clicking the “Download Latest” button on the front page. This will automatically take you to the download page for your operating system.



Once on the download page, just click the option for **Godot 4.5**. This will download the Godot engine to your local computer. From there, unzip the file and simply click on the application launcher – no further installation steps are required!

Download Godot 4 for Windows

**Godot Engine****4.5**

x86_64 - 15 September 2025

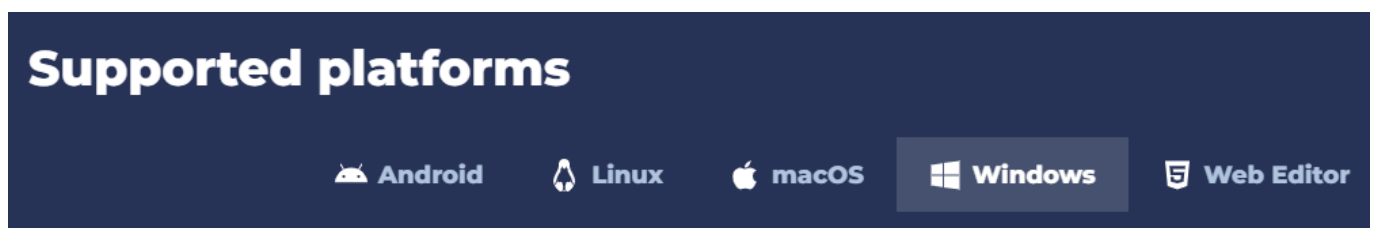
**Godot Engine – .NET****4.5**

x86_64 - C# support - 15 September 2025

Looking for [Godot 3](#), our [experimental releases](#), or a [previous version](#)?



If Godot fails to automatically select the correct operating system for you, you can scroll down to the “Supported platforms” section on the download page to manually select the download version you would like to install:



Before we begin writing any code, it is essential to understand the initial setup of our Godot project. We are starting with a project that already contains a significant amount of material, including all the necessary assets and pre-configured settings. This lesson will guide you through these foundational elements to ensure you have a solid grasp of the project's structure before we proceed.

Project Assets and File System

Our project comes pre-loaded with a variety of assets organized within the file system panel. While there are no scenes yet, the folders are already populated. Let's explore the key folders:

- **Audio:** This folder contains various sound files for the game.
- **Graphics:** This is a crucial folder holding all our visual assets. It is further divided into subfolders for characters, the player, and tilesets. The character and player graphics are designed in a consistent pattern, which will make animations and character swapping easier to implement later. The tilesets, such as those for the overworld and coastline, will be used to build our game world.
- **Global:** This folder contains scripts that are globally accessible throughout the project.

The Global Data Script

Inside the global folder, you will find a file named `data.gd`. This script has been configured as a "global" script (also known as an Autoload in Godot). This means its variables and functions can be accessed from any other script in the game without needing a direct reference. You can verify this by navigating to **Project > Project Settings > Autoload**.

This file serves as a central database for our game and contains several important pieces of data:

- **Enums:** We use enums to define a set of named constants, such as `CharacterStyle` (e.g., blonde, brunette) and `Location` (to identify different areas in the world).
- **Constants:** Simple constant values like `ANIMATION_SPEED` are defined here for easy access and modification.
- **Dictionaries:** A dictionary named `CHARACTER_TEXTURE_DATA` is used to map a character style enum to its corresponding texture file in the file system. This is a clean way to manage character graphics.

While the file also contains data for monsters and attacks, we will only be focusing on the character and location data for now.

Important Project Settings

Several project settings have been pre-configured to establish a baseline for our game. You can review these by navigating to **Project > Project Settings**.

1. **Input Map:** Under the Input Map tab, basic actions have been defined for player movement (left, right, up, down) and for interacting with the world (action).
2. **Display Window:** In the Display > Window section, the game's resolution is set to 1280×720. You are free to change this, as it will not significantly impact the game's logic.
3. **Texture Filtering:** To ensure our pixel art graphics look sharp and not blurry when scaled, the default texture filter has been set to Nearest. You can find this setting under Rendering > Textures.
4. **2D Physics Layers:** Finally, under Layer Names > 2D Physics, three distinct layers have been named: Terrain, Player, and Characters. This will help us manage collisions between different types of game objects.

With this understanding of the project's assets, global data script, and initial settings, we are now



prepared to begin development. Our next step will be to start bringing our characters to life.



In this lesson, we will begin building our game by establishing the foundation for all characters. Both the player and non-player characters (NPCs), such as trainers, share many common attributes like textures, collision detection, movement, and animations. To create an efficient and manageable project, we will create a single parent class, called **Character**, from which both the player and trainers will inherit. We will then create our **Player** class based on this parent.

Setting Up the Level

First, we need a scene to place our characters in. This will serve as the main level environment.

1. Create a new 2D Scene and rename the root node to **Level**.
2. Save this scene as **level.tscn** inside a new folder structure: **scenes/Levels/**.
3. To keep our scene organized, add a **Node2D** named **Background** and another **Node2D** named **Objects** as children of **Level**.
4. Inside the **Objects** node, add one more **Node2D** and name it **Characters**. Your scene tree should look like this:

- **Level (Node2D)**
 - **Background (Node2D)**
 - **Objects (Node2D)**
 - **Characters (Node2D)**

Creating the Base Character Scene

Now, let's create the reusable character scene that will act as a template for our player and trainers.

1. Create a new scene. For the root node, select **CharacterBody2D**.
2. Rename the root node to **Character**.
3. Save the scene as **character.tscn** inside a new folder: **scenes/characters/**.
4. Add a **Sprite2D** node as a child of **Character**. In the Inspector, find the **Texture** property and drag the **character.png** file from **graphics/characters/** into it.
5. The character texture is a spritesheet. To display only the first frame, expand the **Animation** section in the Inspector for the **Sprite2D** and set both **Hframes** and **Vframes** to 4.
6. Add a **CollisionShape2D** node as another child of **Character**. In the Inspector, set its **Shape** property to a new **CapsuleShape2D**. Resize the capsule to roughly fit the character's body.

Creating the Player via Inheritance

With the base **Character** scene complete, we can create the player by inheriting from it. This means our player will automatically have all the nodes and properties we just set up.

1. Go to **Scene > New Inherited Scene...** and select the **character.tscn** file.
2. Rename the root node of this new scene to **Player**.
3. Save the scene as **player.tscn** in the **scenes/characters/** folder.
4. Return to your **Level** scene. Instance the **player.tscn** scene as a child of the **Characters** node.

Scripting with Inheritance

To make our character reusable, we need to set up scripts that can also be inherited. We will use Godot's **class_name** feature to turn our character script into a custom type that the player script can extend.

1. Open the **Character** scene and attach a new script to the root node. Save it as **character.gd**.
2. At the very top of the **character.gd** script, add the **class_name** keyword. This registers "Character" as a new type within Godot.



```
class_name Character extends CharacterBody2D
```

3. Now, open the Player scene. If it has an inherited script icon, right-click the root node and select “Detach Script”.
4. Attach a new script to the Player node, saving it as player.gd.
5. In the new player.gd script, change the line extends CharacterBody2D to extends Character. This ensures the player script inherits all the functionality we will write in character.gd.

```
extends Character
```

Selecting a Character Style

Let’s add a way to change the character’s appearance from the editor. We will use an exported variable and our global data.gd script.

1. In character.gd, declare an exported variable to hold the character’s style. This will create a dropdown in the Inspector based on the CharacterStyle enum in our global script.

```
@export var char_style: Data.CharacterStyle
```

2. Next, add the _ready() function. This code will run when the character enters the scene, loading the correct texture based on the style selected in the Inspector.

```
func _ready() -> void:  
    $Sprite2D.texture = load(Data.character_texture_data[char_style])
```

3. Go back to the Level scene and select the Player node. You may not see the “Character Style” property in the Inspector. This is a common Godot behavior when scripts are changed. To fix this, delete the Player instance from the scene and add it again.
4. With the new Player instance selected, you should now see the **Character Style** property. Select PLAYER from the dropdown menu.

Implementing Movement

Finally, let’s add movement logic. The input will be handled by the Player, but the core movement logic will reside in the base Character class so trainers can use it later.

1. In the Player scene, add a Camera2D node. In the Inspector, set its **Zoom** property to (3, 3) on both X and Y axes. This will make our game view larger.
2. In character.gd, define variables for direction and speed. These will be used by all characters.

```
var direction: Vector2  
var speed = 60
```

3. In player.gd, create a function to capture keyboard input. We use Input.get_vector to easily get a direction vector from the arrow keys or WASD.

```
func get_input():  
    direction = Input.get_vector("left", "right", "up", "down")
```

4. In character.gd, create the move function. This function applies the direction and speed to the character’s velocity and then calls move_and_slide(), which is Godot’s built-in method for moving physics bodies.



```
func move():  
    velocity = direction * speed  
    move_and_slide()
```

5. To make everything work, we need to call these functions every frame. In player.gd, add the `_process` function, which calls `get_input()` to update the direction and `move()` to execute the movement.

```
func _process(delta: float):  
    get_input()  
    move()
```

Adding a Background for Context

If you run the scene now, the player will move, but you won't see it because the background is a solid color. Let's add a simple tilemap to visualize the movement.

1. In the Level scene, select the Background node and add a `TileMapLayer` child.
2. In the Inspector for the `TileMapLayer`, click the empty field next to **Tile Set** and select "New TileSet".
3. Click the new TileSet resource to open the TileSet editor at the bottom of the screen.
4. Drag the `overworld.png` file from the FileSystem dock into the left panel of the TileSet editor. Godot will ask to automatically create tiles; click Yes to create an atlas.
5. Select the `TileMap` node in the scene tree again to open the TileMap editor. Select a grass tile and paint a small area for the player to move on.

Now, run the scene by pressing F6 and selecting the current scene. You should see your player character on a grass background, and you can move it around with the arrow keys. In the next lesson, we will bring the character to life by adding animations.

In this lesson, we will implement a basic animation system for our characters. The characters have two primary states: standing still (idle) and walking. While Godot offers powerful tools like `AnimationPlayer` and `AnimationTree`, they can be overly complex for our simple needs. Instead, we will write a small amount of code to manage these animations effectively.

The Animate Function

Since both the player and the trainers will share the same animation logic, we will implement this functionality within the `Character` class script, `character.gd`. This ensures that any character inheriting from this class will automatically have access to our animation system.

Let's start by creating a new function called `animate`, which will accept the delta time parameter.

```
func animate(delta):  
    pass
```

Understanding the Sprite Sheet

Our character graphics are contained within a sprite sheet. This sheet is organized into a grid. For animation purposes, we are interested in two aspects:

- **Vertical Frames (Rows):** Each row corresponds to a different direction the character is facing. Row 0 is for facing down, 1 for left, 2 for right, and 3 for up.
- **Horizontal Frames (Columns):** Each column represents a single frame of an animation. The first frame (column 0) is the idle pose for that direction. The subsequent frames are for the walking animation.

Our goal is to change the row (`frame_coords.y`) to match the character's direction and cycle through the columns (`frame_coords.x`) to create the walking effect.

Setting the Facing Direction

First, we need to determine which direction the character is facing when they are moving. We can do this by checking if the direction vector has a non-zero value. If it does, we can round its components to get a clear directional vector, like (1, 0) for right or (0, -1) for up.

In `character.gd`, inside the `animate` function, let's add the logic to determine this facing direction and update the sprite's vertical frame.

```
func animate(delta):  
    if direction:  
        var face_dir: Vector2i = Vector2i(round(direction.x), round(direction.y))  
        $Sprite2D.frame_coords.y = Data.character_view_directions[face_dir]
```

Here, we create a `Vector2i` called `face_dir` from the current movement direction. We then use this vector as a key to look up the correct row index from the `character_view_directions` dictionary located in our global `data.gd` script. This index is then assigned to the sprite's `frame_coords.y`, making the character face the correct direction.

To see this in action, we must call our new function from the player. Open `player.gd` and call `animate(delta)` within the `_process` function.

```
func _process(delta: float) -> void:
    get_input()
    if can_move:
        move()
        animate(delta)
    # ... other function calls
```

If you run the game now, you will notice the player character turns to face the direction of movement, although the walking animation is not yet present.

Creating the Walking Animation

To animate the walking cycle, we need to sequentially update the horizontal frame (`frame_coords.x`). We'll add a new variable to the Character class to keep track of the current animation frame.

At the top of `character.gd`, declare the following variable:

```
var current_h_frame: float
```

We use a float because we will be incrementing it by delta time, which is a small fractional number. Using a float ensures we don't lose this precision.

Now, let's update the `animate` function. When the character is moving, we will increase `current_h_frame`. Then, regardless of whether the character is moving or not, we will set the sprite's horizontal frame based on this value.

```
func animate(delta):
    if direction:
        var face_dir: Vector2i = Vector2i(round(direction.x), round(direction.y))
        $Sprite2D.frame_coords.y = Data.character_view_directions[face_dir]
        current_h_frame += Data.ANIMATION_SPEED * delta

    $Sprite2D.frame_coords.x = int(current_h_frame) % $Sprite2D.hframes
```

There are two key parts to this new code:

1. We increment `current_h_frame` by an animation speed constant multiplied by delta.
2. We set `frame_coords.x`. Since this property requires an integer, we convert `current_h_frame` to an int. We also use the modulo operator (%) with the sprite's total number of horizontal frames (`hframes`). This ensures the animation loops correctly, preventing the frame index from going out of bounds.

Handling the Idle State

There is one final issue to address. When the character stops moving, they remain on whatever frame of the walking animation they stopped on. The desired behavior is for them to return to their idle pose (the first frame).

We can fix this by adding an else condition to our check. If there is no direction, it means the

character is standing still, and we should reset the animation frame.

Here is the final version of the animate function in character.gd:

```
func animate(delta):
    if direction:
        var face_dir: Vector2i = Vector2i(round(direction.x), round(direction.y))
        $Sprite2D.frame_coords.y = Data.character_view_directions[face_dir]
        current_h_frame += Data.ANIMATION_SPEED * delta
    else:
        current_h_frame = 0
    $Sprite2D.frame_coords.x = int(current_h_frame) % $Sprite2D.hframes
```

Now, when direction is zero, current_h_frame is set to 0. The subsequent line then sets frame_coords.x to 0, displaying the idle frame. When the character starts moving again, current_h_frame begins incrementing from zero, starting the walking animation smoothly.

With this, our character animation system is complete. The player can now walk around with a proper directional walking animation and will return to a resting pose when stationary.



In addition to the player, our game world will be populated with various trainers. A trainer will typically stand in one place and periodically look around. If a trainer spots the player, they will initiate an interaction, which will eventually lead to a dialogue. In this lesson, we will create the basic trainer character and implement their “look around” behavior.

Creating the Trainer Scene

Since a trainer shares many characteristics with the player, we can reuse our existing Character scene as a base. This is a great use case for scene inheritance.

1. In the FileSystem dock, navigate to the scenes/characters folder.
2. Go to the main menu and select **Scene > New Inherited Scene...**
3. Choose character.tscn as the base scene.
4. In the Scene dock, rename the root node from Character to Trainer.
5. Save this new scene as trainer.tscn in the scenes/characters folder.

Now, let's add an instance of our new trainer to the level. Open your main level scene, instance the trainer.tscn, and place it somewhere on the map. If you run the game, you will see both the player and the stationary trainer.

Enabling YSort

To ensure characters are rendered correctly based on their vertical position (characters lower on the screen appear in front of those higher up), we need to enable YSort on their parent container.

- In your level scene, select the node that contains all your characters (e.g., a node named Characters).
- In the Inspector, find the **Ordering** section and enable the **Y Sort Enabled** property.

With YSort enabled, the engine will automatically handle the draw order of the characters, creating a more convincing sense of depth.

Customizing the Trainer

Just like our player, the trainer's appearance can be customized. Select the trainer instance in your level scene. In the Inspector, you will see the exported Char Style property. You can change this to any of the available styles, such as GREEN, to give your trainer a unique look.

Implementing the “Look Around” Behavior

We want our trainer to randomly change the direction they are facing every few seconds. A Timer node is perfect for this task.

1. Open the trainer.tscn scene.
2. Add a new Node and name it Timers. This will help organize any timers we add.
3. Add a Timer node as a child of Timers and name it WatchTimer.
4. Select the WatchTimer. In the Inspector, set its **Wait Time** to 4.0 seconds and enable **Autostart**.

Trainer Scripting

The trainer requires its own script to handle its unique logic. We will create a new script that extends our base Character class.

1. Select the Trainer root node.



2. In the Inspector, detach the existing character.gd script.
3. Add a new script. In the “Attach Node Script” dialog, ensure it inherits from Character. Godot might default to CharacterBody2D, so you will need to change this manually.
4. Save the script as trainer.gd.

Your new script should start like this:

```
extends Character
```

Next, we need to connect the WatchTimer’s signal. Select the WatchTimer, go to the Node dock, select the **Signals** tab, and connect the timeout() signal to a new function in the Trainer script.

Tracking View Direction

To manage which way a character is looking, we’ll add a new variable to our base Character class. This will be useful for both trainers and the player.

Open character.gd and add the following variable:

```
var view_direction: Vector2i
```

Now, back in trainer.gd, we can implement the logic for when the timer times out. We will pick a random direction and update the sprite’s animation frame accordingly.

```
func _on_watch_timer_timeout() -> void:
    view_direction = [Vector2i.LEFT, Vector2i.RIGHT, Vector2i.UP, Vector2i.DOWN].pick_random()
    $Sprite2D.frame_coords.y = Data.character_view_directions[view_direction]
```

If you run the game now, you will see the trainer change their facing direction every four seconds.

Adding a RayCast for Vision

To detect the player, we will use a RayCast2D node. This node casts a ray in a specific direction and can report any physics bodies it collides with.

1. Open the base character.tscn scene.
2. Add a RayCast2D node as a child of the root Character node.
3. To see the raycast in-game for debugging, go to the top menu and enable **Debug > Visible Collision Shapes**.

We also need to define how far the character can “see”. In character.gd, add a new exported variable:

```
@export var view_distance := 50
```

Now, we can update the raycast’s direction whenever the trainer looks around. Modify the



_on_watch_timer_timeout function in trainer.gd:

```
func _on_watch_timer_timeout() -> void:
    view_direction = [Vector2i.LEFT, Vector2i.RIGHT, Vector2i.UP, Vector2i.DOWN].pick
    _random()
    $Sprite2D.frame_coords.y = Data.character_view_directions[view_direction]
    $RayCast2D.target_position = view_direction * view_distance
```

The target_position of the raycast is now set by multiplying the direction vector by the view distance. When you run the game, you should see the debug line for the raycast rotating with the trainer.

Updating the Player's View Direction

Our player should also have a view direction and a raycast for interactions. Let's refactor the player script to include this functionality.

In player.gd, create a new function called update_view_direction. This function will calculate the direction the player is facing based on their movement input, but it will restrict the direction to the four cardinal directions (up, down, left, right), ignoring diagonals.

```
func update_view_direction():
    if direction:
        var y = round(direction.y) if direction.x == 0 else 0
        view_direction = Vector2i(round(direction.x),y)
        $RayCast2D.target_position = view_direction * view_distance
```

This logic ensures that if the player is moving horizontally (direction.x is not 0), the vertical component of the view direction is forced to 0. If the player is only moving vertically, it uses the rounded y-value.

Call this new function from the _process loop:

```
func _process(delta: float) -> void:
    get_input()
    move()
    animate(delta)
    update_view_direction()
```

Finally, the default view_distance of 50 is a bit too long for the player's interaction range. Select the Player instance in your level scene and, in the Inspector, change its **View Distance** property to 20. Now, the player's interaction raycast is shorter and more appropriate for talking to characters directly in front of them.



In this lesson, we will implement the trainer's behavior for when they spot the player. The goal is for the trainer to notice the player, pause for a moment, and then walk towards them to initiate an interaction. Simultaneously, the player will be stopped and will turn to face the approaching trainer.

Detecting the Player

The first step is to enable the trainer to detect the player. We will use the trainer's RayCast2D node for this. The check will happen continuously within the `_process` function.

To identify the player, we first need to add the player scene to a group. Select the Player node in the Scene dock, go to the Node dock, select "Groups", and add a new group named "Player".

Now, in the trainer.gd script, we can get a reference to the player node and check if the raycast is colliding with it.

```
extends Character
```

```
@onready var player = get_tree().get_first_node_in_group('Player')
```

```
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == player:
        print(player)
```

Configuring Collision Layers

To ensure our detection logic is robust and efficient, it's good practice to properly configure collision layers and masks. This prevents the raycast from detecting unintended objects.

- **Player:** Set its Layer to 2 ("player") and its Mask to detect "terrain" and "characters".
- **Trainer:** Set its Layer to 3 ("characters") and its Mask to detect "terrain" and "player".
- **Trainer's RayCast2D:** Set its Mask to only detect layer 2 ("player").

Triggering the Encounter Only Once

Currently, the detection code in `_process` runs on every frame, which would constantly trigger the encounter. We need this to happen only once. We can manage this with a global state variable. In `data.gd`, declare a new variable to track the character currently involved in an interaction.

```
# In data.gd
var current_char: Character
```

Now, we can update the trainer's script to check this variable. The encounter should only trigger if there isn't already an active character interaction. Once triggered, we set the current trainer as the `current_char`.

```
# In trainer.gd
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == player and not Data.current_char:
        Data.current_char = self
        print(player)
```

With this change, the “player” message will only be printed once when you first enter the trainer’s line of sight.

Adding a Reaction Delay

To make the interaction feel more natural, the trainer will pause briefly before walking towards the player. We can achieve this with a Timer node.

1. Add a Timer node to your trainer scene. You can place it under the existing “Timers” node and name it WalkWaitTimer.
2. In the Inspector, set its **Wait Time** to 0.5 seconds and enable **One Shot**.
3. Connect its timeout signal to a new function in the trainer.gd script.

Now, instead of just printing a message, we will start this timer when the player is detected.

```
# In trainer.gd
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == player and not Data.current_char:
        Data.current_char = self
        $Timers/WalkWaitTimer.start()
```

Calculating the Direction to the Player

When the timer finishes, the trainer needs to know which direction to walk in. We can calculate this using vector math. Since this is a common action for any character, we will add a new function to the parent character.gd script.

```
# In character.gd
func get_char_direction(target_char) -> Vector2i:
    var dir = (target_char.position - position).normalized()
    return Vector2i(round(dir.x), round(dir.y))
```

Back in trainer.gd, we can call this function from our timer’s timeout handler to get the direction towards the player and store it in the trainer’s direction variable.

```
# In trainer.gd
func _on_walk_wait_timer_timeout() -> void:
    direction = get_char_direction(player)
```

Moving and Stopping the Trainer

With the direction calculated, we can make the trainer move. In the _process function, we’ll add logic to move and animate the trainer if it has a non-zero direction.

The trainer should stop upon reaching the player. We’ll define a stopping radius and check the distance to the player. If the trainer is close enough, we’ll reset its direction to zero and stop its walking animation by setting the sprite’s horizontal frame to 0.

```
# In trainer.gd
@export var stop_radius := 24
```



```
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == player and not Data.current_char:
        Data.current_char = self
        $Timers/WalkWaitTimer.start()

    if direction:
        animate(delta)
        if position.distance_to(player.position) < stop_radius:
            direction = Vector2.ZERO
            $Sprite2D.frame_coords.x = 0
            print("showDialog")
            move()
```

Refining Character States

To complete the encounter sequence, we need to adjust the states of both the trainer and the player.

Stopping the Trainer's Random Gazing

Once a trainer spots the player, they should stop looking around randomly. We can control this with a new boolean variable.

```
# In trainer.gd
@export var look_around := true

func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == player and not Data.current_char and look_around:
        Data.current_char = self
        look_around = false
        $Timers/WalkWaitTimer.start()
    # ...

func _on_watch_timer_timeout() -> void:
    if look_around:
        # ... existing random look logic
```

Freezing the Player

During the encounter, the player should not be able to move. We can modify the `get_input` function in `player.gd` to prevent movement when `Data.current_char` is set.

```
# In player.gd
func get_input():
    if not Data.current_char:
        direction = Input.get_vector("left", "right", "up", "down")
        # ...
    else:
        direction = Vector2.ZERO
```



Making the Player Face the Trainer

Finally, the player should turn to face the trainer. We'll create another reusable function in `character.gd` to change a character's view based on a direction vector. Then, we'll call it from the player's script.

```
# In character.gd
func change_view(target: Vector2i):
    $Sprite2D.frame_coords.y = Data.character_view_directions[target]
```

Now, update the player's input logic to call this new function.

```
# In player.gd
func get_input():
    if not Data.current_char:
        # ...
    else:
        direction = Vector2.ZERO
        change_view(get_char_direction(Data.current_char))
```

With these changes, when a trainer spots you, they will pause, walk towards you, and stop. Your character will be frozen in place and will turn to face them, setting the stage for the dialogue and battle to come.



In this lesson, we will begin implementing the dialogue system. Each trainer will have specific lines of dialogue to deliver. We will create the necessary UI elements and scripts to display this text when the player interacts with a trainer or when a trainer spots the player.

Player Interaction

First, we need to enable the player to initiate a conversation with a trainer. This will be done by checking which character the player is facing and pressing an action key.

Open the player.gd script. We will start by adding a variable to keep track of any character the player can interact with.

```
var interactable: Character
```

This interactable variable will be assigned a value using the player's RayCast2D node. Let's create a new function to handle this logic.

```
func check_interactable():
    if $RayCast2D.collide_with_bodies:
        interactable = $RayCast2D.get_collider()
```

This function checks if the raycast is colliding with anything. If it is, the `interactable` variable is set to the collider node, which will be our trainer. If the raycast is not colliding with anything, `interactable` will implicitly become `null`.

We need to call this function continuously. Add the call inside the `\_process` function.

```
func _process(delta: float) -> void:
    get_input()
    move()
    animate(delta)
    update_view_direction()
    check_interactable()
```

Now, we need to handle the player's input to start the dialogue. Inside the `get\_input` function, we will check if the player presses the "action" key while facing an interactable character.

```
func get_input():
    if not Data.current_char:
        direction = Input.get_vector("left", "right", "up", "down")
        if Input.is_action_just_pressed("action") and interactable:
            Data.current_char = interactable
            Data.current_char.change_view(-get_char_direction(Data.current_char))
            Data.current_char.look_around = false
            Data.current_char.show_dialog()
    else:
        direction = Vector2.ZERO
        change_view(get_char_direction(Data.current_char))
```



Let's break down the new code:

- if not Data.current_char: ensures this logic only runs when we are not already in a dialogue or battle.
- if Input.is_action_just_pressed("action") and interactable: checks for the action key press and if there is a valid character to talk to.
- Data.current_char = interactable sets the global current character, effectively pausing player movement and starting the interaction.
- Data.current_char.change_view(-get_char_direction(Data.current_char)) makes the trainer turn and face the player. We get the direction from the trainer to the player and invert it.
- Data.current_char.look_around = false stops the trainer's idle "looking around" behavior.
- Data.current_char.show_dialog() is a function we will create on the trainer to display the dialogue box.

Creating the Dialogue UI

Next, we will create the UI scene for our dialogue box.

1. Create a new scene.
2. Set the root node to be a PanelContainer and rename it to DialogBox.
3. Add a MarginContainer as a child of DialogBox.
4. Add a Label as a child of the MarginContainer. You can leave the text empty for now.
5. Save the scene as dialog_box.tscn inside the scenes/characters folder.

Now, open your trainer.tscn scene and instantiate the DialogBox scene as a child of the trainer. To position it correctly:

1. Select the DialogBox node.
2. In the Inspector, under Layout > Anchor Presets, select **Custom**.
3. Under Layout > Anchor Offsets, adjust the **Bottom** value to move the box up. A value of around -50 should work well.
4. To ensure the dialogue box always appears on top of other elements, go to Ordering in the Inspector and set the **Z Index** to a value like 2.
5. Finally, make the DialogBox invisible by default by unchecking the **Visible** property under the Visibility section.

Displaying the Dialogue

We need a way to show the dialogue box and populate it with text. First, let's create a script for our DialogBox scene.

Attach a new script to the DialogBox root node, named dialog_box.gd. Inside this script, create a function to set the label's text.

```
extends PanelContainer

func set_text(text: String):
    $MarginContainer/Label.text = text
```

Now, open the trainer.gd script. We will add an exported variable to hold the dialogue lines for each trainer.

```
@export var dialog: Array[String]
```

By exporting this variable, you can now select any trainer instance in your level scene and add dialogue strings directly in the Inspector. Add a few test lines for your trainer.

Next, let's create the `show_dialog` function in `trainer.gd`. This function will be responsible for making the dialogue box visible and setting its initial text.

```
func show_dialog():
    $DialogBox.set_text(dialog[0])
    $DialogBox.show()
```

This function calls the `set_text` function we just created in the `DialogBox` script, passing the first string from our dialog array. Then, it makes the dialogue box visible.

With this setup, when you approach a trainer and press the action key, the dialogue box will appear with the first line of text. We also need it to appear if you walk into a trainer's line of sight, so let's update that code as well:

```
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == get_player() and not Data.current_char and look_around:
        Data.current_char = self
        look_around = false
        $Timers/WalkWaitTimer.start()
    if direction:
        animate(delta)
        if position.distance_to(get_player().position) < stop_radius:
            direction = Vector2.ZERO
            $Sprite2D.frame_coords.x = 0
            show_dialog()
        move()
```

We now have the basic framework for our dialogue system. In the next lesson, we will expand upon this to allow the player to advance through multiple lines of dialogue.

With the dialogue box visible and displaying the initial text, the next logical step is to allow the player to cycle through all the available dialogue options. This involves capturing player input and managing the state of the conversation.

Advancing the Dialogue

To progress through the dialogue, we need to listen for player input when a conversation is active. This occurs when `Data.current_char` is assigned a character, which happens in the else block of the player's input handling. We will check if the player presses the "action" key (the space bar by default).

In the `player.gd` script, modify the `get_input` function to call a new method on the current character when the action key is pressed.

```
func get_input():
    if not Data.current_char:
        # ... existing code for movement and initiating dialogue
    else:
        direction = Vector2.ZERO
        change_view(get_char_direction(Data.current_char))
        if Input.is_action_just_pressed("action"):
            Data.current_char.advance_dialog()
```

This new `advance_dialog` method does not exist yet, so we must create it within the `trainer.gd` script.

Implementing the Dialogue Flow

Inside `trainer.gd`, we need a way to track which line of dialogue is currently being displayed. For this, we will add a new variable to store the index of the current dialogue line.

```
var dialog_index: int
```

By default, an integer variable in GDScript is initialized to 0, which is perfect for accessing the first element of our dialogue array. Now, we can create the `advance_dialog` function. This function will check if there are more dialogue lines to show. If so, it will increment the index and update the text in the dialog box. If the conversation is over, it will hide the box and return control to the player.

```
func advance_dialog():
    if dialog_index < dialog.size() - 1:
        dialog_index += 1
        $DialogBox.set_text(dialog[dialog_index])
    else:
        $DialogBox.hide()
        print('start battle')
        Data.current_char = null
```

If you test this now, you can talk to the trainer, press the space bar to see the next line of dialogue, and a final press will hide the box and print "start battle" to the output log. However, you may notice a problem: if you try to talk to the trainer a second time, the dialogue does not progress. This is



because `dialog_index` is not reset after the conversation ends. We can fix this by setting it back to 0 when the dialogue is finished.

```
func advance_dialog():
    if dialog_index < dialog.size() - 1:
        dialog_index += 1
        $DialogBox.set_text(dialog[dialog_index])
    else:
        $DialogBox.hide()
        print('start battle')
        Data.current_char = null
        dialog_index = 0
```

Handling Different Dialogue States

A trainer's dialogue will likely change after they have been defeated in battle. To accommodate this, we can add a new state to our trainer. First, let's add a new exported array for the defeat dialogue and a boolean variable to track the trainer's defeated status.

In `trainer.gd`, add the following variables:

```
@export var defeat_dialog: Array[String]
var defeated := false
```

Now, you can select the Trainer node in the editor and add some text to the new "Defeat Dialog" property in the Inspector.

A System for Testing Defeat

Since we do not have a battle system yet, we need a way to test this new defeated state. We will create a temporary system that marks all trainers as defeated when a specific key is pressed. First, add all trainer characters to a group named "Trainers". You can do this by selecting the trainer node, navigating to the "Node" tab in the inspector, selecting "Groups", and adding the "Trainers" group.

Next, add a simple function in `trainer.gd` to set the defeated state.

```
func test_defeat():
    defeated = true
```

Finally, in `player.gd`, we will add logic to call this function on all trainers when the Tab key (``ui_focus_next``) is pressed.

```
func get_input():
    # ... existing input code

    if Input.is_action_just_pressed("ui_focus_next"):
        for trainer in get_tree().get_nodes_in_group('Trainers'):
            trainer.test_defeat()
```

Displaying Conditional Dialogue

With the state and test system in place, we can now modify our dialogue functions to display the correct text. We will update both `show_dialog` and `advance_dialog`.

In `show_dialog`, use the `defeated` flag to choose the correct dialogue array.

```
func show_dialog():
    $DialogBox.set_text(dialog[0] if not defeated else defeat_dialog[0])
    $DialogBox.show()
```

In `advance_dialog`, we can avoid duplicating code by first determining which dialogue array to use and storing it in a local variable.

```
func advance_dialog():
    var current_dialog = dialog if not defeated else defeat_dialog
    if dialog_index < current_dialog.size() - 1:
        dialog_index += 1
        $DialogBox.set_text(current_dialog[dialog_index])
    else:
        $DialogBox.hide()
        print('start battle')
        Data.current_char = null
        dialog_index = 0
```

Now you can run the game. The trainer will use the default dialogue initially. After you press the Tab key, they will be marked as defeated, and initiating a conversation again will display the defeat dialogue. This creates a more dynamic and responsive interaction system for our game world.



With the dialogue system in place, we can now begin constructing the game's levels. This process will involve creating several different level scenes and a transition system to move between them. A crucial aspect of this is storing trainer data globally so that it persists when we change scenes. We will address these elements step by step, starting with the creation of a modular and visually appealing base level.

Preparing the Base Level

To ensure our levels are modular and easy to create, we will start with a base level scene that will serve as a template. This scene should be relatively empty to provide a clean slate for new, inherited levels.

1. Open your base level.tscn scene.
2. Ensure there are no trainers or other characters besides the player.
3. Select the TileMapLayer node. In the Inspector, clear any existing tiles to make it empty, but do not delete the node itself. Let's ensure this TileMapLayer node is named Floor.
4. Reset the player's position to the origin (0, 0) via the Inspector under Transform.
5. Under the Objects node, add a new Node2D and name it SimpleObjects. This node will contain static objects like houses and trees.

Your base level scene is now prepared to be used as a template.

Creating an Inherited Overworld

We will create new levels by inheriting from our base level.tscn. This allows any changes made to the base level (like the script we'll add later) to automatically apply to all levels that inherit from it.

1. In the Godot editor, go to Scene -> New Inherited Scene.
2. Select level.tscn as the parent scene.
3. Save the new scene as overworld.tscn inside the scenes/levels/ directory.
4. In this new overworld.tscn, select the Floor TileMap node.
5. Using the TileMap panel, paint some grass tiles and sand tiles to create a small area. This will visually distinguish it from the empty base level.

Setting the Main Scene

To test our new level, we need to set it as the default scene that runs when we launch the game.

1. Go to Project -> Project Settings.
2. In the General tab, find the Application -> Run section.
3. Click the folder icon next to the Main Scene property and select your newly created overworld.tscn.

Now, when you run the game, it will load the overworld scene directly.

Animating the Tiles

To make our levels more dynamic, we will animate certain tiles, such as water and coastlines. Since the tileset is part of the base level, these changes must be made in level.tscn.

Water Tiles

1. Open level.tscn and select the Floor TileMap node.
2. Open the TileSet panel at the bottom of the editor.
3. From the FileSystem dock, drag the water.png texture into the TileSet panel. When prompted



to automatically create tiles, select “No”.

4. In the TileSet panel, click on the water texture to open the tile editor. Select the first water tile to add it to the tileset, as well as the last ice tile.
5. With the water tile selected, navigate to the “Animation” section in the Inspector.
6. Set the **Columns** property to 4.
7. Under “Frames”, click “Add Element” and select the next three water tiles in sequence. This creates a four-frame animation.
8. Save the scene. If you return to overworld.tscn and place the water tile, you will see it animates, albeit very slowly.

Coastline Tiles

The coastline tiles will also be animated. The setup is similar, but we will use a script to automate the animation process for every coastline variation.

1. In level.tscn, drag coastline.png into the TileSet panel, again selecting “No” when prompted.
2. The coastline.png texture is arranged so that each row contains a complete animation sequence. The first tile in each row is the base tile.
3. In the tile editor for the coastline, select only the first tile of each row. We do not need to manually set up the animations here.
4. Return to overworld.tscn and place a few coastline tiles next to your water tiles. They will appear static for now.

Automating Animation with GDScript

Manually setting up animations for every coastline tile would be tedious. Instead, we can write a script to handle this automatically. This script will also allow us to control the animation speed for all tiles consistently.

First, let’s add a global constant for the animation speed. Open your global script, data.gd, and add the following constant:

```
const TILE_ANIMATION_SPEED = 2
```

Next, create and attach a new script to the root node of level.tscn, naming it level.gd. This script will configure the tile animations as soon as the level loads.

```
extends Node2D
```

```
func _ready() -> void:
    # tileset animation
    var tileset = $BG/Floor.tile_set as TileSet

    # Coastline animation setup
    var atlas_source = tileset.get_source(2) as TileSetAtlasSource
    for y in atlas_source.get_atlas_grid_size().y:
        for x in int(atlas_source.get_atlas_grid_size().x * 0.25):
            var pos = Vector2i(x * 4, y)
            atlas_source.set_tile_animation_frames_count(pos, 4)
            atlas_source.set_tile_animation_speed(pos, Data.TILE_ANIMATION_SPEED)

    # Water animation setup
    var water_source = tileset.get_source(1)
```



```
water_source.set_tile_animation_speed(Vector2.ZERO, Data.TILE_ANIMATION_SPEED)
```

Code Explanation

Let's break down what this script does:

- It runs in the `_ready` function, which is called when the node enters the scene tree.
- `var tileset = $BG/Floor.tile_set as TileSet`: This line gets the `TileSet` resource from our `Floor TileMap` node.
- `tileset.get_source(2)`: This retrieves the texture source for our coastline tiles. The ID (in this case, 2) corresponds to the order in which the textures were added to the `TileSet`.
- The nested for loops iterate over the coastline texture. The inner loop, for `x` in `int(atlas_source.get_atlas_grid_size().x * 0.25)`, is configured to step through every fourth tile horizontally, targeting the first frame of each animation sequence.
- `var pos = Vector2i(x * 4, y)`: This calculates the coordinate of the base tile for each animation.
- `set_tile_animation_frames_count(pos, 4)` tells Godot that the tile at the specified position has an animation consisting of 4 frames.
- `set_tile_animation_speed(pos, Data.TILE_ANIMATION_SPEED)` sets the animation speed using our global constant.
- Finally, we get the water texture source (ID 1) and set its animation speed to match, ensuring both water and coastline tiles animate in sync. The water tile is at position (0,0), or `Vector2.ZERO`.

After adding this script to `level.tscn`, save it. Now, when you run the `overworld.tscn` scene, you should see both the water and the coastline tiles animating at a consistent, pleasant speed. You have successfully created a modular level system with animated tiles.



At this point in our project, we have established the basic background for our level. However, a level is incomplete without objects for the player to interact with. Our next objective is to populate our scene with various objects, such as trees and other environmental props. Instead of manually adding physics bodies and collision shapes to every single object in the editor, we will implement a more efficient and automated system. We will use GDScript to automatically generate a `StaticBody2D` and an appropriate collision shape for each object sprite when the game starts.

Your Challenge: Automated Collision Generation

Your task is to write a script that processes all sprites located under a parent node (e.g., a “SimpleObjects” node). When the scene is ready, this script should dynamically add the necessary physics components to make these sprites solid objects in the game world.

Requirements

1. **Add Physics Bodies:** For every sprite that is a child of your “SimpleObjects” node, you must add a `StaticBody2D` node as a direct child of that sprite.
2. **Create Collision Shapes:** Inside each newly created `StaticBody2D`, add a `CollisionShape2D` node.
3. **Fit the Shape to the Texture:** The collision shape should accurately match the visible pixels of the sprite’s texture. This ensures that collisions are precise and feel natural.
4. **Implement a Special Case:** You must add a condition to handle specific objects. If a sprite is identified as a “tree” (for instance, by its name), its collision shape should be generated to be significantly thinner than the actual texture, representing just the trunk.

Getting Started

- Begin by populating your “SimpleObjects” node with several different `Sprite2D` nodes in the Godot editor.
- Attach a new GDScript to the “SimpleObjects” parent node.
- The logic for this task should be placed within the `_ready()` function, as this function is executed once the node and all its children have entered the scene tree.
- You will need to iterate through the children of the parent node to access each sprite individually.

This challenge will test your understanding of node creation, scene tree manipulation, and procedural generation in Godot. Take your time to think through the steps and see how far you can get in implementing this automated system.

In this lesson, we will focus on implementing collisions within our game world. Currently, the player can move through all objects and terrain features without restriction. We will address this by adding collision detection in two ways: first, by programmatically adding collision bodies to our sprites, such as trees and houses, and second, by defining collision shapes directly on our tilemap for the terrain.

Enabling Y-Sorting

Before we add collisions, let's enable Y-sorting for our objects. This feature ensures that sprites are drawn in the correct order based on their Y-position, making it appear as though the player is in front of or behind objects correctly. Without this, the player might always appear on top of a tree, regardless of their position.

1. Select the SimpleObjects node in the Scene tree, which is located under the Objects node.
2. In the Inspector panel, find the **Ordering** section.
3. Enable the **Y Sort** property by checking the box.

With Y-sorting enabled, Godot will now render the children of this node based on their vertical position, creating a more convincing sense of depth.

Adding Sprite Collisions via Code

To prevent the player from walking through objects like trees and houses, we need to give them a physical presence. In Godot, this is typically done by adding a StaticBody2D node with a corresponding CollisionShape2D. Since we have many objects, we will automate this process using a script.

We will add this logic to the `_ready` function in our `level.gd` script. This function runs once when the level scene is loaded.

1. Open the `level.gd` script.
2. In the `_ready` function, we will loop through all the child nodes of our SimpleObjects node.
3. For each sprite, we will create and configure the necessary collision nodes.

Here is the initial code to add to your `_ready` function:

```
# sprite collisions
for sprite: Sprite2D in $Objects/SimpleObjects.get_children():
    var texture = sprite.texture
    var shape = RectangleShape2D.new()
    shape.size = texture.get_size()

    var collision_shape = CollisionShape2D.new()
    collision_shape.shape = shape

    var static_body = StaticBody2D.new()
    static_body.add_child(collision_shape)

    sprite.add_child(static_body)
```

Let's break down this code:

- We iterate through each child of `$Objects/SimpleObjects`. We use a type hint (`: Sprite2D`) to let Godot know we are working with sprites.

- We create a new `RectangleShape2D`.
- We set the size of this shape to match the size of the sprite's texture.
- We create a `CollisionShape2D` node and assign our newly created rectangle shape to it.
- We create a `StaticBody2D` node, which provides the physics properties.
- We add the `collision_shape` as a child of the `static_body`.
- Finally, we add the `static_body` as a child of the original sprite, completing the setup.

Refining the Collision Shapes

If you run the game now, you will see that the collisions work, but the collision boxes are quite large, matching the full texture size. This can feel unnatural, especially on the Y-axis where we want the player to be able to overlap with the object slightly. We can adjust the shape's size in our script.

We will also make a special adjustment for trees, which have a lot of transparent space around their trunks. We want their collision shape to be much narrower.

Modify the loop in your `level.gd` script to include these refinements:

```
# sprite collisions
for sprite: Sprite2D in $Objects/SimpleObjects.get_children():
    var texture = sprite.texture
    var shape = RectangleShape2D.new()
    shape.size = texture.get_size()
    shape.size.y *= 0.7
    if 'Tree' in sprite.name:
        shape.size.x *= 0.4

    var collision_shape = CollisionShape2D.new()
    collision_shape.shape = shape

    var static_body = StaticBody2D.new()
    static_body.add_child(collision_shape)

    sprite.add_child(static_body)
```

In this updated code:

- `shape.size.y *= 0.7` reduces the height of all collision shapes to 70% of their original height, allowing for better visual overlap.
- The `if 'Tree' in sprite.name:` condition checks if the sprite's node name contains the word "Tree".
- If it is a tree, `shape.size.x *= 0.4` reduces the width of its collision shape to 40%, making it much easier to navigate around them.

Creating TileMap Collisions

Next, we need to add collisions to our terrain, which is handled by a `TileMap`. This process is done within the Godot editor's `TileSet` editor.

1. Select your `Floor TileMap` node in the Scene tree.
2. In the bottom panel, open the **TileSet** editor.
3. On the right side of the `TileSet` panel, select the **Physics Layers** category and add a new layer by clicking **Add Element**. This will create `Physics Layer 0`.



4. Switch back to the **Paint** category on the left. Below the tile selection, find the **Properties** section and expand **Physics**. Select Physics Layer 0.
5. You can now draw collision shapes on your tiles. Select a tile from your tileset, and a new editor will appear where you can draw polygons. For simple square collisions, you can click the square icon to create a rectangle that fills the tile.
6. Draw collision shapes for all the tiles that should be solid, such as cliffs, walls, and water edges.

To speed up this process, you can use the following tools and shortcuts while the collision editor is active:

- **Grid Snap:** Enable grid snapping for more precise drawing.
- **Keyboard Shortcuts:** Use **R** to rotate the shape, **H** to flip it horizontally, and **V** to flip it vertically. This is very useful for creating corner and edge pieces quickly.

After defining collisions for your tiles, any tiles you paint onto the map will now have those physics properties. Test your level to confirm that the player collides correctly with both the sprites and the terrain.



In this lesson, we will begin implementing the level transition system. The core of this system will involve loading different scenes inherited from our base level scene. To enhance the user experience, we will add a smooth fading transition. We will also ensure the player is placed in the correct starting position after a transition. Let's begin by creating a second level to transition to.

Creating a New Level

Just as we created the overworld, we will create a new level by inheriting from our base level.tscn. This ensures all levels share the same fundamental structure.

1. In the Godot editor, go to **Scene > New Inherited Scene....**
2. Select level.tscn and click **Open**.
3. Save this new scene as house.tscn in the scenes/levels folder.

Next, we need to add a new tileset for the interior of the house. The assets for this are already in the project folder.

1. Select the TileMap node in the house scene.
2. In the Inspector, find the **Tile Set** property.
3. Drag the interior.png file from the FileSystem dock (located in graphics/tilesets) into the Tile Set panel.
4. Godot will prompt you to create an atlas texture. Confirm this action.

You can now use these new tiles to design the interior of the house. Create a simple room with a floor, walls, and a doorway.

Adjusting the Background

If you run the house.tscn scene, you'll notice the background is the default gray color, which doesn't fit our game's aesthetic. We can fix this by modifying the base level script to set a black background for all levels.

Open the level.gd script and add the following line to the _ready function:

```
func _ready() -> void:
    # ... existing code for tileset animation and collisions

    # change bg color
    RenderingServer.set_default_clear_color(Color.BLACK)
```

Now, when you run any level scene, the background will be black, providing a much better look for our interior scene.

Building the Transition System

Our goal is to create a system where the player can move between the overworld and the house by walking through designated doorways. This requires a trigger area and a script to handle the logic.

The Transition Gate Scene

We will create a reusable scene that acts as a portal between levels.

1. Create a new scene with an Area2D node as the root. Rename it to TransitionGate.

2. Add a CollisionShape2D as a child to the TransitionGate.
3. In the Inspector for the CollisionShape2D, set its **Shape** property to a new RectangleShape2D. You can leave the size at the default 20×20 pixels.
4. Save the scene as transition_gate.tscn in the scenes/levels folder.

Now, let's place this gate in our overworld. To keep things organized, first add a new Node2D to your overworld.tscn and name it TransitionGates. Instance your TransitionGate scene as a child of this node and position it in front of the arena's door.

Finally, to ensure the gate only detects the player, select the TransitionGate instance, go to the Inspector, expand the **Collision** section, and set the **Mask** to only layer 2, which is where our player resides.

Stopping the Player

When a transition is triggered, we need to prevent the player from moving. We'll add a new variable and a function to handle this.

First, open the character.gd script and add a new class variable:

```
var can_move := true
```

Next, open the player.gd script. We need to modify the _process function to check if the player is allowed to move.

```
func _process(delta: float) -> void:
    get_input()
    if can_move:
        move()
        animate(delta)
        update_view_direction()
        check_interactable()
```

Now, add a new function to the player.gd script that will stop all movement and prevent further input.

```
func stop():
    direction = Vector2.ZERO
    can_move = false
```

The Fading Transition Layer

To make the transition visually appealing, we'll create a fading effect that covers the screen.

1. Create a new scene with a CanvasLayer as the root node. This ensures our UI element is drawn on top of everything else and is not affected by the game camera.
2. Add a ColorRect as a child of the CanvasLayer.
3. Use the **Layout** menu at the top of the 2D viewport to set the ColorRect's layout to **Full Rect**.



4. In the Inspector, set the **Color** of the ColorRect to black.
5. Rename the root node to TransitionLayer and save the scene.

For this transition layer to be accessible from anywhere in our game, we will set it up as an autoload, or singleton. Go to **Project > Project Settings > Autoload**, select your transition_layer.tscn file, and add it. Make sure the name is TransitionLayer.

Now, attach a script to the TransitionLayer scene and add the following code:

```
extends CanvasLayer
```

```
func _ready() -> void:  
    $ColorRect.modulate.a = 0.0
```

```
func transition(target_location, current_location):  
    var tween = create_tween()  
    tween.tween_property($ColorRect, "modulate:a", 1.0, 0.8)  
    tween.tween_interval(0.5)  
    tween.tween_callback(change_scene.bind(target_location, current_location))  
    tween.tween_property($ColorRect, "modulate:a", 0.0, 0.8)
```

```
func change_scene(target_location, current_location):  
    print(target_location)  
    print(current_location)
```

This script makes the black rectangle invisible by default. The transition function uses a Tween to animate the rectangle's alpha (transparency) to fully opaque, waits for half a second, calls the change_scene function (which currently just prints the locations for testing), and then fades back to transparent.

Triggering the Transition

The final step is to connect everything. Attach a script to your TransitionGate scene. Then, connect the body_entered signal from the Area2D node to this new script. This will create a function that runs whenever a physics body enters the area.

Add the following code to your transition_gate.gd script:

```
extends Area2D
```

```
func _on_body_entered(player: Character) -> void:  
    player.stop()  
    TransitionLayer.transition("house", "overworld")
```

This code calls the stop() function on the player that enters the gate and then starts the global transition animation, passing in our target and current locations as strings for now.

If you run the game and walk into the transition gate, you should see the screen fade to black, the console print "house" and "overworld", and the screen fade back in. The player will also be unable to



move, confirming our system is working as intended. In the next lesson, we will complete the logic to actually change the scene.



In this lesson, we will implement the core logic for transitioning between different scenes in our game. We will begin by creating a system to move from the Overworld to a house and back again. This involves defining locations, loading new scenes, and correctly positioning the player upon arrival.

Setting Up Locations and Scene Paths

To begin, we need a way to exit the house scene, just as we have a way to enter it. In the house.tscn scene, add another TransitionGate node and position it in front of the door.

Next, we need a systematic way to identify our different game locations. We will use an enum for this purpose. In the global data.gd script, modify the Location enum to include the scenes we will be working with for now.

```
# In data.gd
enum Location {OVERWORLD, HOUSE}
```

With our locations defined, we must link these enum values to their corresponding scene files. We can achieve this by creating a dictionary constant that maps each location to its scene path. This makes our scene loading logic much cleaner and easier to manage.

```
# In data.gd
const LEVEL_PATHS = {
    Location.OVERWORLD: "res://scenes/levels/overworld.tscn",
    Location.HOUSE: "res://scenes/levels/house.tscn"
}
```

Implementing the Scene Change

The actual scene switching will be handled by our TransitionLayer. For now, we will implement the direct logic for changing scenes. This process involves several steps:

1. Check if a scene is currently active.
2. If so, free the current scene from memory to prevent memory leaks. This is done using `queue_free()`.
3. Load the new scene resource using the target location and our `LEVEL_PATHS` dictionary.
4. Create an instance of the loaded scene.
5. Add the new scene instance to the scene tree as a child of the root node.
6. Set the new instance as the game's current scene.

Let's add this logic to the `_change_scene` function in `transition_layer.gd`. Note that the function will require parameters to know which scene to load (`target_loc`) and which scene we are coming from (`current_loc`).

```
# In transition_layer.gd
func _change_scene(target_loc: Data.Location, current_loc: Data.Location):
    if get_tree().current_scene:
        get_tree().current_scene.queue_free()
    var scene = load(Data.LEVEL_PATHS[target_loc]).instantiate()
    get_tree().root.add_child(scene)
    get_tree().current_scene = scene
```

Note, after setting this up, make sure the transition function also has type checking in the parameters as well to keep our code consistent and secure:

```
func transition(target_loc: Data.Location, current_loc: Data.Location):  
    // Function content
```

Making Transitions Dynamic

Currently, our transition gates are not configured to specify a destination. To make them dynamic, we will add exported variables to the transition_gate.gd script. This will allow us to set the target and current locations for each gate directly in the Godot editor.

```
# In transition_gate.gd  
extends Area2D  
  
@export var target: Data.Location  
@export var current: Data.Location  
  
func _on_body_entered(player: Character) -> void:  
    player.stop()  
    TransitionLayer.transition(target, current)
```

With these variables exposed, you can now configure each gate:

- **Overworld Gate:** Set its Target to HOUSE and its Current to OVERWORLD.
- **House Gate:** Set its Target to OVERWORLD and its Current to HOUSE.

Positioning the Player on Entry

When we transition, you'll notice the player always appears at the origin point (0, 0) of the new scene. We need to set a specific starting position. We will create a function in our level.gd script that finds the correct entry gate and positions the player near it.

This function, `player_start_position`, will accept the location the player is coming *from* as an argument. It will then iterate through all transition gates in the level to find the one that leads back to that previous location. Once found, it will set the player's position accordingly.

```
# In level.gd  
func player_start_position(target: Data.Location):  
    for gate in $TransitionGates.get_children():  
        if gate.target == target:  
            $Objects/Characters/Player.position = gate.position + Vector2(0, 30)
```

To use this function, we need to call it from our TransitionLayer after the new scene has been loaded. We also need a global variable in data.gd to keep track of the player's current location.

```
# In data.gd
```

```
var current_loc: Data.Location
```

Now, we can update `_change_scene` in `transition_layer.gd` to call our new positioning function and update the global location.

```
# In transition_layer.gd
func _change_scene(target_loc: Data.Location, current_loc: Data.Location):
    if get_tree().current_scene:
        get_tree().current_scene.queue_free()
    var scene = load(Data.LEVEL_PATHS[target_loc]).instantiate()
    get_tree().root.add_child(scene)
    get_tree().current_scene = scene

    scene.player_start_position(current_loc)
    Data.current_loc = target_loc
```

Customizing the Spawn Offset

The hardcoded position offset is not ideal. We can improve this by creating a function within `transition_gate.gd` that provides a configurable offset. We will add an exported enum to select a direction and an exported variable for the distance.

```
# In transition_gate.gd
@export_enum("Up", "Down", "Left", "Right") var start_offset = "Up"
@export var distance := 30

# ...

func get_start_offset() -> Vector2:
    return {
        "Up": Vector2.UP,
        "Down": Vector2.DOWN,
        "Right": Vector2.RIGHT,
        "Left": Vector2.LEFT
    }[start_offset] * distance
```

Now, update the `player_start_position` function in `level.gd` to use this new, more flexible method.

```
# In level.gd
func player_start_position(target: Data.Location):
    for gate in $TransitionGates.get_children():
        if gate.target == target:
            $Objects/Characters/Player.position = gate.position + gate.get_start_offset()
```

Remember to configure the Start Offset direction for each gate in the editor. For the overworld gate leading into the house, "Down" is appropriate. For the house gate leading out, "Up" is suitable.



Setting the Player's View Direction

As a final touch, let's ensure the player is facing the correct direction when a new scene loads. We can easily calculate the view direction from the start offset vector. By normalizing the offset, we get a unit vector representing the direction, which we can pass to the player's `change_view` function.

```
# In level.gd
func player_start_position(target: Data.Location):
    for gate in $TransitionGates.get_children():
        if gate.target == target:
            $Objects/Characters/Player.position = gate.position + gate.get_start_offset()
            var view_dir = gate.get_start_offset().normalized() as Vector2i
            $Objects/Characters/Player.change_view(view_dir)
```

With this final addition, our scene transition system is now functional. The player can move between scenes, appearing at the correct location and facing the appropriate direction.

In this lesson, we will address a significant issue in our game's mechanics: the persistence of data across different scenes. Currently, when the player transitions between scenes, any changes to a character's state, such as a Trainer being defeated, are lost. This happens because Godot reloads scenes from scratch, resetting all nodes to their default values. We will implement a system to store Trainer data globally, ensuring their state is preserved throughout the game.

Preliminary Bug Fixes

Before we can implement the data storage system, we must resolve a couple of errors that arise from reloading scenes. These errors occur because references to nodes that were freed during a scene transition become invalid.

First, an error occurs when trying to determine a character's direction towards a target that no longer exists. To fix this, we will add a check in the `get_char_direction` function within the `character.gd` script. This ensures the code only runs if both the character and its target are valid.

```
func get_char_direction(target_char) -> Vector2i:
    if target_char and self:
        var dir = (target_char.position - position).normalized()
        return Vector2i(round(dir.x), round(dir.y))
    else:
        return Vector2i.DOWN
```

Second, the Trainer's reference to the player node becomes invalid after a scene change. Instead of storing a direct reference that can be freed, we will create a function to fetch the player node whenever it's needed. In `trainer.gd`, replace the `@onready var player` line with a new function:

```
func get_player():
    return get_tree().get_first_node_in_group('Player')
```

You must then update any code that used the player variable to call this new `get_player()` function instead. For example, in the `_process` function:

```
func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == get_player() and not Data.current_char and look_around:
        # ...
    if direction:
        # ...
        if position.distance_to(get_player().position) < stop_radius:
            # ...
        # ...
```

With these fixes, transitioning between scenes should no longer produce errors, allowing us to focus on data persistence.

Storing Trainer Data Globally

Our strategy is to use a global dictionary to store the state of each Trainer. This dictionary will live in

our global singleton, `data.gd`, ensuring it persists across all scenes.

First, open `data.gd` and add a new variable to hold the character data:

```
var char_data = {  
  
}
```

Next, we need a way to uniquely identify each Trainer instance in the game. A combination of the scene name and the node name provides a reliable unique ID. In `trainer.gd`, create a function to generate this ID.

```
func get_unique_id() -> String:  
    var current_scene_name = get_owner().scene_file_path.get_file().get_basename()  
    var node_name = name  
    return current_scene_name + "_" + node_name
```

Saving and Loading State

Now we can use this unique ID to save and load the Trainer's state. We will use the `_enter_tree` function, which is called when a node enters the Scene Tree, making it the perfect place to initialize its state from our global data store.

In `trainer.gd`, add a new variable for the unique ID and implement the `_enter_tree` function:

```
var unique_id: String  
  
func _enter_tree() -> void:  
    unique_id = get_unique_id()  
    if unique_id not in Data.char_data:  
        Data.char_data[unique_id] = {'defeated': defeated}  
    else:  
        defeated = Data.char_data[unique_id]['defeated']
```

This logic works as follows:

- When a Trainer enters the scene, it generates its unique ID.
- It then checks if an entry for this ID already exists in the global `Data.char_data` dictionary.
- If no entry exists, it's the first time we've encountered this Trainer. We create a new entry and save its current (default) defeated status.
- If an entry does exist, it means we have seen this Trainer before. We then load the saved defeated status from the global dictionary and apply it to the current Trainer instance.

Updating the Global State

The final step is to ensure that whenever a Trainer's state changes (i.e., they are defeated), the global data store is updated. We can achieve this elegantly by using a setter function for the defeated variable.



Modify the defeated variable declaration in trainer.gd as follows:

```
var defeated := false:
    set(value):
        defeated = value
        Data.char_data[unique_id]['defeated'] = value
```

Now, any time the defeated variable is assigned a new value (e.g., defeated = true), the setter function will automatically execute. It updates the local variable and immediately saves the new value to our global Data.char_data dictionary, using the Trainer's unique ID.

With this system in place, you can test the functionality. Engage a Trainer in dialogue, defeat them, and then leave the area. Upon your return, the Trainer should remember their defeated state and provide the appropriate dialogue, demonstrating that our data persistence system is working correctly.



You have now acquired all the basic skills required to create levels for your game. To put this knowledge into practice, you are encouraged to take on a creative challenge. Spend some time designing and building several visually appealing levels to solidify your understanding and begin constructing your game world.

Your Level Design Challenge

The primary goal of this exercise is to apply the techniques you have learned to build a variety of complete and polished levels. The more you practice, the more intuitive the process will become. Focus on the following key requirements for your creations.

Core Requirements

- **Create Multiple Levels:** Try to build a whole bunch of different levels. This will give your game a sense of scale and provide diverse environments for the player to explore.
- **Incorporate Animations:** Bring your world to life by including animations. These can be applied to characters, environmental objects, or special effects to make your scenes more dynamic.
- **Use Varied Terrains:** Do not limit yourself to a single style. Make sure to design and integrate a couple of different terrain types to add visual interest and variety.
- **Add Trainers:** Populate your levels with a whole bunch of trainers. These characters will make your world feel inhabited and provide the foundation for future gameplay interactions.
- **Implement Collisions:** For any interior parts of your levels, you will need to add collisions to prevent the player from walking through walls and furniture. This is a crucial step in making your environments feel solid and believable.

Looking Ahead

At present, our focus is on adding trainers to the levels. However, as we continue to build the project, we will introduce other non-player characters (NPCs), such as nurses and quest characters. The levels you design now will serve as the foundation for these future additions, so building them thoughtfully is important.

Take this opportunity to be creative and build a nice-looking game. Good luck!

In this lesson, we will focus on polishing the user interface and fixing a critical bug. Previously, several levels were created, including the overworld, a house, a hospital, and two arenas. These levels have been populated with trainers, each having unique dialogue for their default and defeated states. While creating detailed levels can be time-consuming, it is a crucial part of game development. Now, we will enhance the visual presentation of our dialogue and address a gameplay issue.

Styling the Dialog Box

Currently, the dialog box is a plain, unstyled panel. To improve the game's aesthetic, we will apply a custom style. We will perform these changes in the editor on our ``DialogBox`` scene.

1. **Center Text:** Select the ``Label`` node within the ``DialogBox``. In the Inspector, set both the Horizontal and Vertical Alignment properties to "Center". This ensures the text is always centered within the box.
2. **Remove Default Panel Style:** Select the root ``PanelContainer`` node. In the Inspector, navigate to **Theme Overrides > Styles**. Create a new ``StyleBoxEmpty`` for the "Panel" property. This will make the default background of the panel invisible. We use a ``PanelContainer`` because it automatically resizes to fit the ``Label``'s content.
3. **Add a Custom Frame:** Add a ``NinePatchRect`` node as a child of the ``PanelContainer``. This node is ideal for creating scalable frames. Assign your frame texture (e.g., ``Frame.png`` from the ``UI`` folder) to its ``Texture`` property. Set the **Patch Margin** for Left, Top, Right, and Bottom to 5. This ensures the corners of the frame do not stretch when the box resizes. To ensure the frame appears behind the text, make sure the ``NinePatchRect`` is ordered before the ``MarginContainer`` in the scene tree.
4. **Set Font Color:** With the ``PanelContainer`` still selected, go to **Theme Overrides > Colors**. Set the "Font Color" property to black to ensure readability against the new frame.
5. **Apply a Custom Font:** Navigate to **Theme Overrides > Fonts**. Load your custom font file (e.g., ``dogicapixel.otf``) into the "Font" property.
6. **Add Margins:** The text may appear too close to the edges of the frame. To fix this, select the ``MarginContainer`` node. Under **Theme Overrides > Constants**, set the margin properties for Left, Top, Right, and Bottom to 10. This will create a small padding around the text.

After these changes, when you interact with a trainer, the dialog box will appear much more polished and integrated with the game's overall style.

Fixing a Trainer Interaction Bug

A bug has been identified where trainers in the arena levels behave incorrectly. When the player approaches a trainer, the trainer turns in the wrong direction, which prevents the dialogue and battle sequence from triggering correctly. This happens because our script gets a reference to the player node when the scene loads, but this reference can become invalid or point to the wrong object if the player moves between scenes.

The solution is to get a fresh reference to the player node right when it's needed. We will modify the ``trainer.gd`` script to fix this.

In the ``_on_walk_wait_timer_timeout`` function, the code originally used a variable that was set when the trainer was initialized. We will change it to call the ``get_player()`` function instead, which actively finds the current player node in the scene tree.

```
func _on_walk_wait_timer_timeout() -> void:
    direction = get_char_direction(get_player())
```



This small change ensures that the trainer always knows the player's correct position, regardless of the level. The interaction will now work as intended, with the trainer turning to face the player correctly before initiating dialogue.

With the user interface improved and a key bug resolved, our game world is much more robust.



In the first part of this course, we have successfully built the foundational elements of our game's overworld. Now that this initial phase is complete, let's review what we have accomplished and look ahead to the exciting features we will be implementing next.

Course Recap: Building the Overworld

We have established a robust and functional overworld by implementing several key systems. The features we have created so far include:

- **Overworld Creation:** We utilized a tilemap layer node and the principle of inheritance to construct our game world. This approach also allows for the easy creation of additional levels.
- **Trainers and Global Data:** We have populated our world with numerous trainers. To manage their information effectively, we implemented a global data storage system, ensuring that trainer data remains consistent across different scenes.
- **Dialog System:** A basic dialog system was developed to enable interactions within the game.

What's Next: Implementing the Battle System

With the overworld in place, our next major goal is to develop the battle system. This will be a core component of our game, where you will be able to:

- Create a variety of unique monsters.
- Implement combat mechanics to make them fight each other.
- Engage in battles with either trainers or wild monsters you encounter in the world.

About Zenva

Zenva is an educational platform trusted by over a million students, offering a wide range of courses for both beginners and those looking to expand their skills. Our courses are designed to be versatile, allowing you to learn in the way that suits you best. You can:

- Watch comprehensive video tutorials.
- Read concise lesson summaries to reinforce your knowledge.
- Follow along with the instructor using the included project files.

We look forward to seeing you in the course!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

data.gd

Found in folder: **/global**

This code defines several enums for character styles, locations, monsters, elements, attacks, menu states, and biomes. It also defines constants for animation speeds, character view directions, character texture data, level paths, monster data, attack data, and element modifiers. The code also declares variables for the current character, current location, and character data.

extends Node

```
enum CharacterStyle {CHARACTER, BLOND, PLAYER, GREEN, FIRE, GRASS, ICE, PURPLE, STRAW, BOY, GIRL}
enum Location {OVERWORLD, HOUSE, HOSPITAL, GRASS_ARENA, FIRE_ARENA}
enum Monster {SPARCHU, ATROX, JACANA, CINDRILL, CLEAF, DRAEM, FINIETTE, FINSTA, FRIOLERA, GULFIN, IVIERON, LARVEA, PLUMA, PLUMETTE, POUCH, VULKEO}
enum Element {FIRE, WATER, PLANT}
enum Attack {CLAW, FIRE, ICE, WATER, HEAL, EXPLOSION}
enum MenuState {MAIN, ATTACK, DEFEND, SWAP, CATCH, SELECT}
enum Biome {GRASS, DESERT, ICE}
```

```
const ANIMATION_SPEED = 6
const TILE_ANIMATION_SPEED = 2
const character_view_directions = {
    Vector2i.DOWN: 0,
    Vector2i.LEFT: 1,
    Vector2i.RIGHT: 2,
    Vector2i.UP: 3,
    Vector2i.ZERO: 0,
    Vector2i(1,1) : 2, # down right
    Vector2i(1,-1) : 2, # up right
    Vector2i(-1,1) : 1, # down left
    Vector2i(-1,-1) : 1, # up left
}
const character_texture_data = {
    CharacterStyle.PLAYER: "res://graphics/characters/player.png",
    CharacterStyle.GREEN: "res://graphics/characters/green.png",
    CharacterStyle.BLOND: "res://graphics/characters/blond.png",
    CharacterStyle.CHARACTER: "res://graphics/characters/character.png",
    CharacterStyle.FIRE: "res://graphics/characters/fire_boss.png",
    CharacterStyle.GRASS: "res://graphics/characters/grass_boss.png",
    CharacterStyle.ICE: "res://graphics/characters/ice_boss.png",
    CharacterStyle.STRAW: "res://graphics/characters/straw.png",
    CharacterStyle.PURPLE: "res://graphics/characters/purple.png",
    CharacterStyle.BOY: "res://graphics/characters/young_boy.png",
    CharacterStyle.GIRL: "res://graphics/characters/young_girl.png",
}
const LEVEL_PATHS = {
```

```
Location.OVERWORLD: "res://scenes/levels/overworld.tscn",
Location.HOUSE: "res://scenes/levels/house.tscn",
Location.HOSPITAL: "res://scenes/levels/hospital.tscn",
Location.GRASS_ARENA: "res://scenes/levels/plant_arena.tscn",
Location.FIRE_ARENA: "res://scenes/levels/fire_arena.tscn"
}
const monster_data = {
  Monster.SPARCHU: {
    'name': 'Sparchu',
    'battle texture': "res://graphics/battle sprites/Sparchu.png",
    'icon texture': "res://graphics/menu sprites/Sparchu.png",
    'stats': {'max hp': 10, 'max ep': 2, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.CINDRILL: {
    'name': 'Cindrill',
    'battle texture': "res://graphics/battle sprites/Cindrill.png",
    'icon texture': "res://graphics/menu sprites/Cindrill.png",
    'stats': {'max hp': 14, 'max ep': 12, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.VULKEO: {
    'name': 'Vulkeo',
    'battle texture': "res://graphics/battle sprites/Vulkeo.png",
    'icon texture': "res://graphics/menu sprites/Vulkeo.png",
    'stats': {'max hp': 17, 'max ep': 14, 'speed': 1},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.GULFIN: {
    'name': 'Gulfin',
    'battle texture': "res://graphics/battle sprites/Gulfin.png",
    'icon texture': "res://graphics/menu sprites/Gulfin.png",
    'stats': {'max hp': 9, 'max ep': 8, 'speed': 0.8},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.WATER},
  Monster.ATROX: {
    'name': 'Atrox',
    'battle texture': "res://graphics/battle sprites/Atrox.png",
    'icon texture': "res://graphics/menu sprites/Atrox.png",
    'stats': {'max hp': 12, 'max ep': 8, 'speed': 2},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
  Monster.JACANA: {
    'name': 'Jacana',
    'battle texture': "res://graphics/battle sprites/Jacana.png",
    'icon texture': "res://graphics/menu sprites/Jacana.png",
    'stats': {'max hp': 3, 'max ep': 0.4, 'speed': 4},
    'attacks': {0: Attack.CLAW, 12: Attack.FIRE, 24: Attack.HEAL, 30: Attack.EXPLOSION}
  },
  'element': Element.FIRE},
```

```
}
const attack_data = {
  Attack.CLAW: {
    'name': 'Claw',
    'offensive': true,
    'cost': 10,
    'amount': 10,
    'element': Element.PLANT,
    'texture': "res://graphics/attack effects/attack7.png"},
  Attack.FIRE: {
    'name': 'Fire',
    'offensive': true,
    'cost': 12,
    'amount': 15,
    'element': Element.FIRE,
    'texture': "res://graphics/attack effects/attack5.png"},
  Attack.WATER: {
    'name': 'Splash',
    'offensive': true,
    'cost': 9,
    'amount': 11,
    'element': Element.WATER,
    'texture': "res://graphics/attack effects/attack3.png"},
  Attack.ICE: {
    'name': 'Ice shards',
    'offensive': true,
    'cost': 13,
    'amount': 20,
    'element': Element.WATER,
    'texture': "res://graphics/attack effects/attack1.png"},
  Attack.EXPLOSION: {
    'name': 'Explode',
    'offensive': true,
    'cost': 20,
    'amount': 100,
    'element': Element.FIRE,
    'texture': "res://graphics/attack effects/attack4.png"},
  Attack.HEAL: {
    'name': 'Heal',
    'offensive': false,
    'cost': 15,
    'amount': -30,
    'element': Element.PLANT,
    'texture': "res://graphics/attack effects/attack6.png"}}
const element_modifier = {
  Element.FIRE: {Element.FIRE: 1, Element.WATER: 0.5, Element.PLANT: 2},
  Element.WATER: {Element.FIRE: 2, Element.WATER: 1, Element.PLANT: 0.5},
  Element.PLANT: {Element.FIRE: 0.5, Element.WATER: 2, Element.PLANT: 1}}

var current_char: Character
var current_loc: Data.Location
var char_data = {
```

```
}
```

character.gd

Found in folder: **/scenes/characters**

This code defines a Character class that extends the CharacterBody2D class. It includes variables for character style, view distance, direction, speed, current horizontal frame, view direction, and a flag for movement ability. The code also includes functions for initializing the character's texture, moving the character based on direction and speed, animating the character's sprite based on direction and delta time, getting the direction towards a target character, and changing the character's view direction.

```
class_name Character extends CharacterBody2D

@export var char_style: Data.CharacterStyle
@export var view_distance := 50
var direction: Vector2
var speed = 60
var current_h_frame: float
var view_direction: Vector2i
var can_move := true

func _ready() -> void:
    $Sprite2D.texture = load(Data.character_texture_data[char_style])

func move():
    velocity = direction * speed
    move_and_slide()

func animate(delta):
    if direction:
        var face_dir: Vector2i = Vector2i(round(direction.x), round(direction.y))
        $Sprite2D.frame_coords.y = Data.character_view_directions[face_dir]
        current_h_frame += Data.ANIMATION_SPEED * delta
    else:
        current_h_frame = 0
    $Sprite2D.frame_coords.x = int(current_h_frame) % $Sprite2D.hframes

func get_char_direction(target_char) -> Vector2i:
    if target_char and self:
        var dir = (target_char.position - position).normalized()
        return Vector2i(round(dir.x), round(dir.y))
    else:
        return Vector2i.DOWN

func change_view(target: Vector2i):
    $Sprite2D.frame_coords.y = Data.character_view_directions[target]
```

dialog_box.gd

Found in folder: **/scenes/characters**

This code defines a custom Godot node that extends the PanelContainer class. It includes a `set_text` function that takes a string parameter and sets the text of a Label node inside a MarginContainer. The `$` symbol is used to access nodes in the scene tree relative to the current node.

```
extends PanelContainer
```

```
func set_text(text: String):  
    $MarginContainer/Label.text = text
```

player.gd

Found in folder: **/scenes/characters**

This code defines a player character in a game that can move, animate, and interact with other characters. The player's movement is controlled by user input, and it can change its view direction based on the direction it's moving. The player can also initiate interactions with other characters when the "action" button is pressed, and can advance through dialogues when the same button is pressed again.

```
extends Character
```

```
var interactable: Character
```

```
func _process(delta: float) -> void:  
    get_input()  
    if can_move:  
        move()  
        animate(delta)  
        update_view_direction()  
        check_interactable()
```

```
func get_input():  
    if not Data.current_char:  
        direction = Input.get_vector("left", "right", "up", "down")  
        if Input.is_action_just_pressed("action") and interactable:  
            Data.current_char = interactable  
            Data.current_char.change_view(-get_char_direction(Data.current_char))  
            Data.current_char.look_around = false  
            Data.current_char.show_dialog()  
    else:  
        direction = Vector2.ZERO  
        change_view(get_char_direction(Data.current_char))  
        if Input.is_action_just_pressed("action"):  
            Data.current_char.advance_dialog()  
  
    if Input.is_action_just_pressed("ui_focus_next"):  
        for trainer in get_tree().get_nodes_in_group('Trainers'):
```

```
trainer.test_defeat()

func update_view_direction():
    if direction:
        var y = round(direction.y) if direction.x == 0 else 0
        view_direction = Vector2i(round(direction.x),y)
        $RayCast2D.target_position = view_direction * view_distance

func check_interactable():
    if $RayCast2D.collide_with_bodies:
        interactable = $RayCast2D.get_collider()

func stop():
    direction = Vector2.ZERO
    can_move = false
```

trainer.gd

Found in folder: **/scenes/characters**

This code defines a trainer character in a game that can interact with the player. The trainer has properties such as dialog, defeat dialog, and a unique ID. The trainer can detect when the player is nearby, display dialog, and initiate a battle when the dialog is finished.

```
extends Character

@onready var player = get_tree().get_first_node_in_group('Player')
@export var stop_radius := 24
@export var look_around := true
@export var dialog: Array[String]
@export var defeat_dialog: Array[String]
var dialog_index: int
var defeated := false:
    set(value):
        defeated = value
        Data.char_data[unique_id]['defeated'] = value
var unique_id: String

func test_defeat():
    defeated = true

func get_player():
    return get_tree().get_first_node_in_group('Player')

func _enter_tree() -> void:
    unique_id = get_unique_id()
    if unique_id not in Data.char_data:
        Data.char_data[unique_id] = {'defeated': defeated}
```

```
else:
    defeated = Data.char_data[unique_id]['defeated']

func get_unique_id() -> String:
    var current_scene_name = get_owner().scene_file_path.get_file().get_basename()
    var node_name = name
    return current_scene_name + "_" + node_name

func _process(delta: float) -> void:
    if $RayCast2D.get_collider() == get_player() and not Data.current_char and look_around:
        Data.current_char = self
        look_around = false
        $Timers/WalkWaitTimer.start()
    if direction:
        animate(delta)
        if position.distance_to(get_player().position) < stop_radius:
            direction = Vector2.ZERO
            $Sprite2D.frame_coords.x = 0
            show_dialog()
            move()

func _on_watch_timer_timeout() -> void:
    if look_around:
        view_direction = [Vector2i.LEFT, Vector2i.RIGHT, Vector2i.UP, Vector2i.DOWN].pick_random()
        $Sprite2D.frame_coords.y = Data.character_view_directions[view_direction]
        $RayCast2D.target_position = view_direction * view_distance

func _on_walk_wait_timer_timeout() -> void:
    direction = get_char_direction(get_player())

func show_dialog():
    $DialogBox.set_text(dialog[0] if not defeated else defeat_dialog[0])
    $DialogBox.show()

func advance_dialog():
    var current_dialog = dialog if not defeated else defeat_dialog
    if dialog_index < current_dialog.size() - 1:
        dialog_index += 1
        $DialogBox.set_text(current_dialog[dialog_index])
    else:
        $DialogBox.hide()
        print('start battle')
        Data.current_char = null
        dialog_index = 0
```

level.gd

Found in folder: **/scenes/levels**

This code sets up tileset animations for the floor and water tiles, creates collision shapes for simple objects like trees, changes the background color to black, and positions the player character at a specific location based on the target gate.

```
extends Node2D
```

```
func _ready() -> void:
    # tileset animation
    var tileset = $BG/Floor.tile_set as TileSet
    var atlas_source = tileset.get_source(2) as TileSetAtlasSource
    for y in atlas_source.get_atlas_grid_size().y:
        for x in int(atlas_source.get_atlas_grid_size().x * 0.25):
            var pos = Vector2i(x * 4, y)
            atlas_source.set_tile_animation_frames_count(pos, 4)
            atlas_source.set_tile_animation_speed(pos, Data.TILE_ANIMATION_SPEED)
    var water_source = tileset.get_source(1)
    water_source.set_tile_animation_speed(Vector2.ZERO, Data.TILE_ANIMATION_SPEED)

    # sprite collisions
    for sprite: Sprite2D in $Objects/SimpleObjects.get_children():
        var texture = sprite.texture
        var shape = RectangleShape2D.new()
        shape.size = texture.get_size()
        shape.size.y *= 0.7
        if 'Tree' in sprite.name:
            shape.size.x *= 0.4
        var collision_shape = CollisionShape2D.new()
        collision_shape.shape = shape
        var static_body = StaticBody2D.new()
        static_body.add_child(collision_shape)
        sprite.add_child(static_body)

    # change bg color
    RenderingServer.set_default_clear_color(Color.BLACK)
```

```
func player_start_position(target: Data.Location):
    for gate in $TransitionGates.get_children():
        if gate.target == target:
            $Objects/Characters/Player.position = gate.position + gate.get_start_offset()
            var view_dir = gate.get_start_offset().normalized() as Vector2i
            $Objects/Characters/Player.change_view(view_dir)
```

transition_gate.gd

Found in folder: **/scenes/levels**

This code defines an Area2D node that triggers a transition when a player character enters it. It specifies variables for the target and current locations, a start offset direction, and a distance value. When the player enters the area, the character is stopped, and the TransitionLayer transitions to the target location from the current location.

```
extends Area2D
```

```
@export var target: Data.Location
@export var current: Data.Location
@export_enum("Up", "Down", "Left", "Right") var start_offset = "Up"
@export var distance := 30
```

```
func _on_body_entered(player: Character) -> void:
    player.stop()
    TransitionLayer.transition(target, current)
```

```
func get_start_offset() -> Vector2:
    return {
        "Up": Vector2.UP,
        "Down": Vector2.DOWN,
        "Right": Vector2.RIGHT,
        "Left": Vector2.LEFT
    }[start_offset] * distance
```

transition_layer.gd

Found in folder: **/scenes/levels**

This code defines a scene transition effect. It fades a `ColorRect` node to black, changes the current scene to a new one based on the target location, and then fades the `ColorRect` back to transparent. The `\_change\_scene` function handles freeing the current scene, loading and adding the new scene, setting the player's start position, and updating the `Data.current\_loc` variable.

```
extends CanvasLayer
```

```
func _ready() -> void:
    $ColorRect.modulate.a = 0.0
```

```
func transition(target_loc: Data.Location, current_loc: Data.Location):
    var tween = create_tween()
    tween.tween_property($ColorRect, "modulate:a", 1.0, 0.8)
    tween.tween_interval(0.5)
    tween.tween_callback(_change_scene.bind(target_loc, current_loc))
    tween.tween_property($ColorRect, "modulate:a", 0.0, 0.8)
```

```
func _change_scene(target_loc: Data.Location, current_loc: Data.Location):
    if get_tree().current_scene:
        get_tree().current_scene.queue_free()
    var scene = load(Data.LEVEL_PATHS[target_loc]).instantiate()
    get_tree().root.add_child(scene)
    get_tree().current_scene = scene
    scene.player_start_position(current_loc)
    Data.current_loc = target_loc
```